Sentencias de control en PL/SQL

Dónde estamos

- Ya escriben SQL y bloques anónimos. Falta el paso que convierte una consulta en un programa: decidir y repetir.
- Todo lo que viene después del curso, funciones, procedimientos y disparadores, se apoya en estas cuatro páginas de sintaxis.

Qué vamos a hacer

- Recorrer las tres familias de control con su sintaxis exacta de 19c, no con la versión aproximada de otros lenguajes.
- Decidir cuál estructura corresponde a cada problema y sustentar la elección.
- Provocar los errores típicos en la sesión, leer el código ORA y corregirlos.

Recorrido

- Condicionales: IF, CASE y la lógica de tres valores. Iteración: LOOP, WHILE, FOR y FOR sobre cursor.
- Control secuencial y por qué casi nunca se usa. Procesamiento masivo como alternativa al bucle fila a fila.
- Taller guiado paso a paso con el script 01 y cinco ejercicios con criterio de autoevaluación.

Lo que hay que tener fresco

El bloque y sus secciones

- DECLARE es opcional, BEGIN y END no lo son. La sección EXCEPTION cierra el bloque y no interrumpe la estructura de control que la contiene.
- Las sentencias de control viven en la sección ejecutable; no se pueden usar en la sección declarativa.

Dos motores en la misma sesión

- El motor PL/SQL ejecuta el código procedimental y entrega cada sentencia SQL al motor SQL. Cada entrega es un cambio de contexto con costo.
- Un bucle que ejecuta un INSERT por vuelta paga ese costo en cada iteración. Es la razón técnica detras de FORALL y BULK COLLECT.

Salida por pantalla

- DBMS_OUTPUT.PUT_LINE escribe en un buffer de sesión; sin SET SERVEROUTPUT ON no se ve nada y el código no falla.
- Sirve para aprender y para depurar, no para registrar eventos en producción: para eso va una tabla de log.

Las tres familias de sentencias de control

La documentación 19c organiza el control de flujo en tres grupos, y solo dos importan en el día a día

Familia	Sentencias	Cuándo corresponde
Selección condicional	IF THEN / IF THEN ELSE / IF THEN ELSIF / CASE simple / CASE buscado	Ejecutar sentencias distintas segun el valor o la condición evaluada
Iteración	LOOP / WHILE LOOP / FOR LOOP numérico / FOR LOOP sobre cursor	Repetir el mismo tratamiento sobre una serie de valores o de filas
Control secuencial	GOTO / NULL	Saltar a una etiqueta o dejar constancia explícita de que no se hace nada

EXIT y CONTINUE no son una familia aparte
Son modificadores del flujo dentro de un bucle: EXIT lo termina, CONTINUE cierra la iteración en curso. Ambos aceptan la cláusula WHEN, que evita envolverlos en un IF.

Regla de lectura: si el código necesita un comentario para explicar por que salta, la estructura elegida no era la correcta.

IF: tres formas, una sola regla de cierre

```
-- 1. IF THEN: una sola rama
IF v_nota >= 3 THEN v_estado := 'APROBADO'; END IF;

-- 2. IF THEN ELSE: dos caminos excluyentes
IF v_nota >= 3 THEN v_estado := 'APROBADO';
ELSE               v_estado := 'REPROBADO';
END IF;

-- 3. IF THEN ELSIF: cascada evaluada en orden, se detiene en la primera verdadera
IF    v_nota >= 4 THEN v_nivel := 'ALTO';
ELSIF v_nota >= 3 THEN v_nivel := 'MEDIO';
ELSE                   v_nivel := 'BAJO';
END IF;
```

Dos detalles que cuestan puntos en el parcial

Se escribe ELSIF, sin E intermedia: ELSEIF y ELSE IF abren un IF nuevo que exige su propio END IF. Y el cierre siempre es END IF, con espacio, nunca ENDIF.

El orden de los ELSIF es lógica, no estilo: la primera condición verdadera gana y las demas ni se evaluan.

La condición no es booleana de dos valores: es de tres

Una condición en PL/SQL puede valer TRUE, FALSE o NULL, y NULL se comporta distinto en cada estructura

Expresion	Resultado	Consecuencia práctica
v_nota = NULL	NULL	La rama THEN no se ejecuta, aunque la variable si sea nula
v_nota IS NULL	TRUE o FALSE	Es la única forma correcta de preguntar por ausencia de valor
NOT (condición NULL)	NULL	Negar un NULL no lo convierte en TRUE
NULL AND FALSE	FALSE	El AND puede resolverse sin conocer el operando nulo
NULL OR TRUE	TRUE	El OR también puede resolverse; no ocurre lo mismo con AND y TRUE

Dónde aparece en la práctica
Un WHILE cuya variable de control nunca se inicializo no entra al bucle ni una vez, y no hay error que lo delate. Inicialice siempre, o use NVL en la condición cuando el valor nulo sea legitimo.

CASE simple y CASE buscado

El simple compara un selector por igualdad; el buscado evalua condiciones independientes

```
-- CASE simple
CASE v_jornada
  WHEN 'D' THEN v_txt := 'Diurna';
  WHEN 'N' THEN v_txt := 'Nocturna';
  ELSE          v_txt := 'Sin definir';
END CASE;
```

```
-- CASE buscado
CASE
  WHEN v_nota >= 4 THEN v_txt := 'Alto';
  WHEN v_nota >= 3 THEN v_txt := 'Medio';
  ELSE                  v_txt := 'Bajo';
END CASE;
```

El CASE simple no sirve para comparar contra NULL

El selector se compara con el operador de igualdad, y NULL nunca es igual a NULL. Si el valor nulo es un caso válido del negocio, use CASE buscado con IS NULL o atrápelo en la rama ELSE.

Una cascada de más de tres ELSIF sobre la misma variable casi siempre se lee mejor como CASE.

Sentencia CASE y expresion CASE

Misma palabra reservada, dos construcciones distintas: una ejecuta, la otra devuelve un valor

```
-- SENTENCIA: cada WHEN lleva sentencias
CASE
  WHEN v_n >= 3 THEN v_e := 'APROBADO';
  ELSE               v_e := 'REPROBADO';
END CASE;
```

```
-- EXPRESION: cada WHEN devuelve un valor
v_e := CASE
         WHEN v_n >= 3 THEN 'APROBADO'
         ELSE               'REPROBADO'
       END;
```

	Sentencia CASE	Expresion CASE
Cierre	END CASE;	END; sin la palabra CASE
Sin ELSE y sin coincidencia	Levanta CASE_NOT_FOUND, ORA-06592	Devuelve NULL en silencio
Donde se usa	En el flujo del bloque	En una asignación, un parámetro o una consulta SQL

Cómo elegir entre IF y CASE

Use IF cuando

- Las condiciones son heterogéneas y tocan variables distintas: IF v_saldo > 0 AND v_estado = 'A' THEN ...
- Necesita aprovechar la evaluación en corto circuito, por ejemplo validar que un valor no es nulo antes de operarlo.

Use CASE cuando

- Todas las ramas responden a la misma pregunta sobre el mismo dato. El lector entiende la intención sin leer cada condición.
- Necesita el resultado como valor: ahi la expresion CASE reemplaza tres lineas de asignación por una.

Lo que decide en código de producción

- Legibilidad primero: quien mantiene el código dentro de dos años rara vez es quien lo escribio.
- Una sola salida por estructura. Anidar IF dentro de IF dentro de IF es la señal de que falta extraer una función.
- Las ramas exhaustivas se documentan solas: incluya siempre ELSE, aunque sea para levantar un error explícito.

LOOP básico: se repite hasta que usted lo detenga

```
v_i := 1;
<<principal>>
LOOP
  DBMS_OUTPUT.PUT_LINE('Iteración ' || v_i);
  v_i := v_i + 1;
  EXIT principal WHEN v_i > 5;   -- sin esta linea, bucle infinito
END LOOP principal;
```

Característica	Consecuencia
La condición se evalua dentro del cuerpo	El bloque se ejecuta al menos una vez, siempre
La salida es responsabilidad del programador	EXIT, EXIT WHEN o una excepcion; no hay salida implícita
Admite etiqueta	EXIT etiqueta abandona ese bucle, util cuando hay anidamiento

Escriba el EXIT en el mismo momento en que escribe el LOOP, no después: así no se olvida.

WHILE: la condición se evalua antes de entrar

```
v_saldo  := 1000;
v_cuotas := 0;
WHILE v_saldo > 0 LOOP
  v_saldo  := v_saldo - 250;
  v_cuotas := v_cuotas + 1;
END LOOP;
-- si v_saldo llega nulo, el cuerpo no se ejecuta ninguna vez
```

Diferencia con el LOOP básico

El LOOP básico garantiza una ejecución; el WHILE puede ejecutarse cero veces. Elija segun la semántica del problema: cobrar la primera cuota siempre no es lo mismo que cobrar solo si hay saldo.

Condición con varios operandos

WHILE v_i <= v_tope AND v_encontrado = 'N' es correcto, pero si cualquiera de los dos puede ser nulo el bucle se detiene sin aviso. Normalice con NVL o valide antes de entrar.

FOR LOOP numérico: el índice lo declara la sentencia

```
FOR i IN 1..5 LOOP          -- i no se declara ni se asigna
  DBMS_OUTPUT.PUT_LINE(i);
END LOOP;

FOR i IN REVERSE 1..5 LOOP  -- recorre 5,4,3,2,1; los límites se escriben igual
  NULL;
END LOOP;
```

Regla del índice	Qué ocurre si se incumple
Es de solo lectura dentro del bucle	Asignarle un valor produce error de compilacion, PLS-00363
Existe solo entre LOOP y END LOOP	Referenciarlo afuera no compila, o toma otra variable del mismo nombre
Los límites se evaluan una sola vez, al entrar	Cambiar la variable del límite adentro no altera el número de vueltas
Si el límite inferior supera al superior	El cuerpo no se ejecuta y no se levanta ninguna excepcion

FOR sobre cursor: el bucle que más se usa en producción

Recorre el resultado de una consulta sin declarar el cursor, sin abrirlo y sin cerrarlo

```
FOR r IN (SELECT id_estudiante, nombre, nota
            FROM ctl_estudiante
           WHERE nota IS NOT NULL
           ORDER BY nota DESC) LOOP
  DBMS_OUTPUT.PUT_LINE(r.nombre || ' -> ' || r.nota);
END LOOP;
```

Qué hace Oracle por usted

Declara el registro r con la estructura de la consulta, abre el cursor, hace FETCH en cada vuelta, detecta el fin de datos y cierra el cursor incluso si el bucle termina por una excepcion.

Si dentro del bucle ejecuta un UPDATE por cada fila, ya no esta ante un problema de control sino de procesamiento masivo.

CONTINUE y EXIT con etiquetas

```
<<externo>>
FOR i IN 1..10 LOOP
  CONTINUE externo WHEN MOD(i, 2) = 0;   -- omite los pares
  <<interno>>
  FOR j IN 1..3 LOOP
    EXIT externo WHEN i * j > 12;        -- sale de los dos bucles
    DBMS_OUTPUT.PUT_LINE(i || ' x ' || j);
  END LOOP interno;
END LOOP externo;
```

El riesgo real de CONTINUE

En un LOOP básico, un CONTINUE colocado antes del incremento del contador salta justamente la linea que hace avanzar el bucle: la sesión queda girando sin fin. En un FOR LOOP no ocurre, porque el índice lo maneja Oracle.

Cuándo el bucle deja de ser la respuesta

Fila a fila contra procesamiento masivo: la misma lógica, dos costos distintos

```
-- fila a fila: un cambio de contexto por vuelta
FOR r IN (SELECT id, nota FROM ctl_nota) LOOP
  UPDATE ctl_nota
     SET nota = r.nota * 1.05
   WHERE id = r.id;
END LOOP;
```

```
-- masivo: una sola entrega al motor SQL
SELECT id BULK COLLECT INTO v_ids
  FROM ctl_nota;
FORALL k IN 1..v_ids.COUNT
  UPDATE ctl_nota SET nota = nota * 1.05
   WHERE id = v_ids(k);
```

Criterio de ingenieria, no de moda

BULK COLLECT trae el resultado a memoria de sesión: sobre volumenes grandes se acota con LIMIT. Y si la operación se puede expresar como un solo UPDATE en SQL puro, esa sigue siendo la opción más rápida de las tres.

GOTO y NULL: el par que casi no se usa

```plsql
-- GOTO salta a una etiqueta del mismo ámbito
IF v_error = 'S' THEN
  GOTO fin;
END IF;
<<fin>>
NULL;   -- la etiqueta debe preceder a una sentencia
```

```plsql
-- NULL documenta la decisión de no actuar
CASE v_tipo
  WHEN 'A' THEN procesar_a;
  WHEN 'B' THEN procesar_b;
  ELSE NULL;   -- los demas tipos no se procesan
END CASE;
```

La peor práctica que compila sin quejarse

WHEN OTHERS THEN NULL; silencia cualquier error del bloque. El programa sigue como si nada hubiera pasado y el diagnóstico se pierde. Si hay que ignorar un error, ignore uno concreto y deje registro.

Errores frecuentes y cómo se leen

Síntoma	Causa típica	Corrección
La sesión no responde	LOOP sin EXIT, o CONTINUE antes del incremento	Mover el incremento o usar FOR LOOP
ORA-06592 CASE_NOT_FOUND	Sentencia CASE sin ELSE y ningún WHEN coincide	Agregar ELSE, aunque sea para levantar un error propio
PLS-00363	Asignación al índice del FOR LOOP	Usar una variable auxiliar declarada en el bloque
El bucle no ejecuta ninguna vuelta	Variable de control nula o límites invertidos	Inicializar antes de entrar y verificar el rango
La rama nunca se cumple	Comparación con = NULL en vez de IS NULL	Reescribir la condición o normalizar con NVL

Cómo trabajar el script 01 paso a paso

Antes de ejecutar

- Abra 01_control_plsql19c.sql en SQL Developer o SQL*Plus con un usuario que tenga CREATE TABLE y CREATE PROCEDURE.
- Ejecute SET SERVEROUTPUT ON. Sin esa linea los ejemplos corren pero usted no ve ninguna salida.

Durante la práctica

- El script esta numerado por pasos. Cada paso dice qué se va a hacer, qué debe observar y cuál es el resultado esperado.
- No ejecute el archivo completo de una vez: avance paso por paso y compare la salida con lo que anuncia el comentario.
- Los pasos marcados como error provocado deben fallar. Lea el código ORA antes de pasar a la corrección que viene abajo.

Al terminar

- Resuelva los cinco ejercicios finales. Cada uno trae pista y criterio de logrado para que usted mismo se evalue.
- Ejecute el bloque de limpieza solo cuando haya terminado: borra las tablas de práctica del esquema.

Lo que hay que llevarse

Del criterio

- La estructura correcta es la que hace evidente la intención: cascada de condiciones con CASE, número conocido de vueltas con FOR.
- Toda condición es de tres valores. Inicialice, o pregunte con IS NULL.
- Un bucle que ejecuta DML fila a fila es un problema de rendimiento esperando ocurrir.

Del oficio

- El EXIT se escribe junto con el LOOP. El ELSE se escribe junto con el CASE. Las dos omisiones tienen nombre y número de error.
- Ningún GOTO sobrevive a una revisión de código seria: si lo necesita, levante una excepcion.

Lecturas para llegar preparado a la próxima sesión

- PL/SQL Language Reference 19c, capitulo PL/SQL Control Statements, secciones de IF, CASE y LOOP.
- Database Development Guide 19c, capitulo Controlling Program Flow, para la versión narrada con ejemplos.
- docs.oracle.com/en/database/oracle/oracle-database/19/lnpls/plsql-control-statements.html